

Six attempts at quasi-polynomial unknot recognition: synthesis, cross-validation, and a faster exact recognizer

Synthesis report prepared with Claude Code

17–18 September 2026

Abstract

Six independently produced archives each tried to implement the $n^{O(\log n)}$ unknot recognition algorithm announced in Marc Lackenby’s February 2021 talk. This report extracts them into numbered directories, reads their reports, reviews their code, re-runs their test suites, and cross-validates them against each other on shared inputs. All six reach the same honest conclusion: the quasi-polynomial algorithm was not implemented, because the accelerated hierarchy operations named on the last slide have no published algorithmic specification with cost bounds. What they deliver instead is consistent and correct as far as we can test: five reduced-Khovanov recognizers agree on 44 shared inputs, six ball-pattern testers agree on 2000 random spherical patterns, and the grid-diagram search of archive 01 agrees with Khovanov homology on every rectangular diagram we checked. Building on their ideas we add a seventh recognizer, **fastunknot**, whose exact part is a scanning computation of Khovanov homology (Bar-Natan’s local algorithm with delooping and Gaussian elimination) preceded by an Alexander-polynomial filter. It is still exponential in the worst case, but its cost is governed by the width of a scan through the diagram rather than by the crossing number: the 80-crossing unknot family on which every cube implementation needs 2^{80} states is decided in eleven milliseconds. Nine further runs then proposed accelerations of that recognizer. We extracted and measured all nine on one machine, integrated every idea with a measurable effect into version 0.2 (the input that timed out at 600 seconds now takes under a second, and recognition of the Conway knot went from 59 ms to 1.3 ms), recorded a per-idea ablation, and wrote a Rust implementation that is faster again by an order of magnitude. The $n^{O(\log n)}$ target remains open; Section 5 states precisely what is missing, and nothing in the accelerations touches it.

Contents

1	Task, sources, and outcome	3
1.1	What was asked	3
1.2	The talk	3
1.3	The 2026 preprint	4
1.4	Outcome in one paragraph	4
2	The six archives	4
2.1	Extraction	4
2.2	What each archive delivers	5
2.3	Common structure of the six reports	5
3	Code review	6

3.1	The Khovanov recognizers (02–06)	6
3.2	The pattern testers (all six)	6
3.3	Normal coordinates and potentials	7
3.4	The grid search of archive 01	7
4	Cross-validation	7
4.1	Five Khovanov implementations on shared inputs	7
4.2	Six pattern testers on random spherical patterns	8
4.3	Grid search versus Khovanov homology	8
4.4	What the cross-validation does and does not show	9
5	What is missing for $n^{O(\log n)}$	9
6	A faster exact recognizer: fastunknot 0.1	9
6.1	Design	9
6.2	The scanning computation	10
6.3	Validation	11
6.4	Measurements	11
6.5	Limits of the new package	12
7	Nine acceleration proposals	12
7.1	What was delivered	12
7.2	The ideas	12
7.3	Same-machine comparison	13
8	fastunknot 0.2	14
8.1	What was adopted	14
8.2	Ablation	15
8.3	Complexity: what changed and what did not	16
8.4	Verification	16
9	A Rust implementation	16
9.1	Design	16
9.2	Verification	17
9.3	Profile	17
9.4	Where the three versions stand	18
10	A third round: constant factors in Python, and how to measure them	18
10.1	What changed	18
10.2	Results	19
10.3	Measured and not adopted	23
10.4	An episode about measurement	23

11 A port to Bend 2, and what its parallelism delivered	24
11.1 Setting	24
11.2 What was ported, and how it was checked	24
11.3 What the fork delivers	25
11.4 Where the time goes	25
11.5 Verdict	26
12 Assessment and recommendations	26
A Files produced	27

1 Task, sources, and outcome

1.1 What was asked

The original request to each of the six runs was an implementation of the main theorem of the supplied talk [1]: an algorithm that decides whether an n -crossing knot diagram represents the unknot in time $n^{O(\log n)}$. The present task was to unpack the six resulting archives, read their reports, review their code, and synthesize what was achieved and what obstacles remain. A second task was added mid-way: write the asymptotically fastest algorithm possible, trying to reach $n^{O(\log n)}$. A third task followed: unpack nine proposals for accelerating that algorithm, compare their ideas, incorporate those that give a measurable speedup, and write an optimized Rust implementation.

1.2 The talk

The supplied file `docs/quasipolynomial-talk.pdf` has 109 pages including progressive-reveal duplicates. Its logical content is short:

- pp. 11–14: the theorem, $n^{O(\log n)}$ for an n -crossing diagram;
- pp. 15–30: hierarchies, boundary patterns, essential patterns (a proper disc meeting the pattern at most three times must cut off an empty disc, an arc, or a tripod), and the Waldhausen–Johannson characterization: the knot is nontrivial iff the exterior admits an essential hierarchy ending in balls;
- pp. 38–53: simplification of an inessential hierarchy by a violating disc (compress or pattern-compress the last surface the disc runs over, discard later surfaces), and the basic algorithm as a flowchart;
- pp. 54–65: termination by a lexicographic complexity $(g(S_1), \dots, g(S_\ell))$, with the warning that “genus” may mean χ_- or a pattern-sensitive variant, and the estimate “running time at most Lg^L ” if every surface has complexity at most g and hierarchies have length at most L ;
- pp. 66–74: the four speed-ups needed for $n^{O(\log n)}$: (1) surfaces with $g \leq O(n^2)$, (2) compressed hierarchy encodings (Agol–Hass–Thurston), (3) multi-surfaces, (4) generalised Heegaard splittings and Cheeger regions, “the effect of this is to ensure that $L \leq O(\log n)$ ”;
- pp. 75–108: sketches of each speed-up, including the Cheeger-region condition $g(\partial M_i) \leq \frac{1}{3} \min\{g(H_i), g(H) - g(H_i)\}$;
- p. 109: the final flowchart with the boxes BUILD HIERARCHICAL MULTI-SURFACE, CUT ALONG SURFACE, USE CHEEGER REGION, HAKEN’S LEMMA, WEAKLY REDUCE, SIMPLIFY MULTI-SURFACE, and the terminal test “does the graph dual to the boundary pattern have a violating cycle?”.

The slides describe the strategy; they contain no data structures, no algorithms for the boxes, and no proof of the depth bound.

1.3 The 2026 preprint

All six archives located Lackenby’s July 2026 preprint [2] (“Incompressible surfaces, hierarchies and unknot recognition”, 54 pages). We downloaded it and read Section 9 (“The number of steps”) ourselves. It opens: “There are various steps in the algorithm from Theorem 1.3 that are not described with enough precision to be able to estimate their running time. So, we will initially content ourselves with an analysis of the number of times that the algorithm goes through any of the steps”, in terms of the maximal hierarchy length L and the maximal pattern-complexity g , and “At present, we will not attempt to bound these two quantities.” Proposition 9.1 then gives the iteration bound $L(g+1)^L$ using exactly the base- $(g+1)$ digit argument that the talk sketches in base g . Section 10 refines the algorithm to obtain a bound on L that is *linear* in the initial q -complexity of the handle structure, not logarithmic in n . The preprint also supplies two facts the archives use: Proposition 2.6 (a pattern on a 3-ball is essential iff it is empty, a circle, or a connected graph whose planar dual is simple and 4-connected or K_3 or K_4) and Definition 3.4 (pattern-complexity $\chi_P(S) = -4\chi(S) + |S \cap P|$).

So the accelerated, logarithmic-depth algorithm of the talk is, as of September 2026, announced but not specified in any source available to us. This is the single obstacle behind every “partial” verdict below, and it is mathematical, not an engineering shortfall.

1.4 Outcome in one paragraph

None of the six archives, and not the new package either, implements a recognizer with a proved $n^{O(\log n)}$ bound. Each archive contains a complete exact recognizer with an exponential bound, plus genuinely implemented polynomial pieces of the hierarchy strategy (the terminal ball-pattern test, cocycle-dual normal coordinates, the potential arithmetic), and each report says so without hedging. We verified every test suite, cross-validated the archives against each other and against independent computations, found no incorrect verdict, and identified one gap in a correctness argument (Section 3.4). The new package **fastunknot** improves the exact part substantially in practice (Section 6); its worst case is still $2^{O(n)}$.

2 The six archives

2.1 Extraction

The six ZIP files in **reports/** were extracted into **reports/01** through **reports/06**, each with the single wrapper directory removed, in the order of the archive modification times:

Dir	archive	package	Python lines	tests
01	unknot_recognition_implementation.zip	unknot (grid search)	1753	40
02	unknot_recognition_partial_implementation.zip	unknotlab	1939	64
03	unknot_workbench.zip	unknot_lab	1431	49
04	unknot_recognition_partial.zip	unknot	1422	62
05	unknot_recognition_partial_implementation (1).zip	unknot_recognition	1882	58
06	unknot-recognition-implementation.zip	unknot	1278	45

Every archive contains a README, a LaTeX report with PDF, examples, recorded results, and a standard-library-only Python package. All six test suites were re-run here on CPython 3.14.4 under Windows (the archives recorded CPython 3.13.5 on Linux) and pass with exactly the claimed number of tests.

2.2 What each archive delivers

Archive 01: rectangular diagrams and Dynnikov moves. The only archive that does not use Khovanov homology. It represents knots as grid diagrams, searches the finite space of grids reachable by non-increasing moves (exchanges and destabilizations, modulo cyclic shifts) and relies on Dynnikov’s monotonic simplification theorem [5]: an unknot reaches the 2×2 grid, a knot is certified by a state set closed under all moves. Certificates (positive path, non-unit determinant, closed set) are replayed by a verifier that enumerates moves independently. Bound $2^{O(g \log g)}$ in the grid size g ; the report is careful that g is not the crossing number. It validated all 44 719 connected grids of size ≤ 6 (1285 shift orbits). It also contains the pattern test, exact rational H^1 of a simplicial complex, cocycle-dual normal coordinates, and the potential arithmetic.

Archive 02: Khovanov cube with fast certificates. Descending-diagram test (a diagram traversed meeting every crossing first on its over-strand is trivial), Fox determinant, then the full reduced Khovanov complex over $\mathbb{F}_2$. Its normal-surface module accepts genuine face-pairing triangulations, checks orientability and sphere/disc vertex links, and computes cocycle-dual normal coordinates without expanding discs. Six braid/determinant pairs were checked against Wolfram KnotData.

Archive 03: workbench with an explicit exponential lower bound. Khovanov cube with default resource ceilings (65 536 states), a bond-enumeration pattern test with checkable witnesses, and a `bound` command for the conditional $L(g+1)^L$ arithmetic. Its report proves that its own implementation is not quasi-polynomial: on the unknot $\sigma_1 \widehat{\sigma_2 \cdots \sigma_n}$ the cube has 2^n states and 3^n generators, and the determinant filter cannot help since the determinant is 1.

Archive 04: Khovanov cube with Knot Atlas fixtures. Khovanov cube, dual-4-connectivity pattern test, potential arithmetic. It checks $10_{124} = T(3, 5)$ from an Atlas PD code and contains a clean chain-level proof that the unreduced $\mathbb{F}_2$ complex is two copies of the reduced one, which justifies reading reduced ranks off published unreduced tables.

Archive 05: Khovanov cube with Reidemeister preprocessing. Crossing-decreasing Reidemeister I/II moves with replayable traces, determinant filter, Khovanov cube; bond-enumeration pattern test; simplicial normal coordinates with manifold validation; the pattern-complexity $-4\chi + |S \cap P|$ of the preprint. It exhausts all 682 two-strand words of odd length ≤ 9 and gives an independent braid-theoretic proof that its eight-crossing “hard unknot” is trivial.

Archive 06: Khovanov cube with 11-crossing tests. Khovanov cube, bond-enumeration pattern test, a `verify` command that recomputes a report with $d^2 = 0$ checks. Its fixtures include the Kinoshita–Terasaka and Conway knots (Alexander polynomial 1, reduced rank 33). Its source audit notes that a 34-page Oxford version of the slides uses the factor $1/10$ instead of $1/3$ in the Cheeger condition and displays $O((\log n)^2)$ for the hierarchy length; substituting that depth into the digit count would give $n^{O((\log n)^2)}$.

2.3 Common structure of the six reports

Read side by side, the reports agree on every substantive point, which is itself evidence since they were produced independently:

1. The requested bound is not established; the delivered recognizer is a different, exponential algorithm; resource limits return UNKNOWN and never a verdict.

2. The Khovanov route is justified by Kronheimer–Mrowka [3] plus a universal-coefficient argument: $\dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(K; \mathbb{F}_2) \geq \text{rank } \widetilde{\text{Kh}}(K; \mathbb{Z})$, and the rational reduced homology is nonzero because its Euler characteristic is the reduced Jones polynomial, so $\mathbb{F}_2$ -rank one forces rational rank one.
3. Determinant one is never a positive certificate ($T(3, 5)$ and the Kinoshita–Terasaka knot are the standard witnesses).
4. The talk’s “base g ” digit argument needs base $g + 1$ when the digit g is allowed, since $(1, 0)$ and $(0, g)$ have the same base- g value; all six use base $g + 1$, as does Proposition 9.1 of the preprint.
5. The terminal pattern test is implemented only under an explicit “already a 3-ball” hypothesis, and $b_1 = 0$ is not ball recognition ($S^2 \times I$ is the standard counterexample).
6. The missing operations are listed by name: initial layered handle structure of the exterior, compressed cutting with provenance, hierarchical multi-surfaces with the pattern-sensitive quadratic bound, lifting a violating dual cycle to a disc in an earlier stage, Cheeger-region detection and Heegaard modification, Haken’s lemma and weak reduction, and an end-to-end bit-cost proof including restarts.

3 Code review

3.1 The Khovanov recognizers (02–06)

All five have the same architecture, evidently converged upon rather than copied (variable names, data layouts and gradings differ): enumerate the 2^n resolutions with a disjoint-set structure, encode a reduced generator as a bit mask over the unmarked circles with the marked circle fixed to x , build each differential column as a Python integer (XOR arithmetic over $\mathbb{F}_2$), and compute ranks by pivot elimination. Points checked in the review:

- Input validation is uniformly careful: one component, spherical rotation system via $V - E + F = 2$ on the ribbon graph (so virtual diagrams are rejected, not misclassified), labels normalized, `{"pd": []}` defined as one circle.
- The Frobenius algebra tables ($m(x \otimes x) = 0$, $\Delta(1) = 1 \otimes x + x \otimes 1$, $\Delta(x) = x \otimes x$) are identical everywhere, and every implementation checks that the marked circle keeps its x label.
- $d^2 = 0$ is available as an exhaustive optional check in all five and is exercised by their tests.
- Gradings are deliberately unnormalized; only the total rank is used, so writhe shifts are irrelevant. This is correct for recognition.
- Resource semantics are consistent (UNKNOWN, exit code 2 or 3; the codes differ between archives, which is a minor interoperability wart).
- Archive 03 imposes default ceilings, so its default invocation is incomplete above 16 crossings; the README says so.

We found no correctness problem. The determinant filters (Fox colouring matrix, Bareiss elimination with exact-division checks) are also equivalent.

3.2 The pattern testers (all six)

Three different criteria are implemented for “is this cubic spherical pattern essential on a ball”: dual simple and 4-connected with the K_3 , K_4 exceptions (01, 04), no dual loop, parallel pair or non-facial dual triangle (02), and no bond of size 1 or 2 and no three-edge bond with more than one vertex on each side (03, 05, 06). Each report proves its own criterion from the definition. The proofs are correct; the bond argument in archive 06 (the cycle rank of a side with k vertices and r cut edges is $(k - r + 2)/2$) is the cleanest. Input conventions differ (darts $2e, 2e + 1$; signed edge ends; positional darts; edge identifiers occurring twice), which is why a converter was needed for cross-validation.

3.3 Normal coordinates and potentials

Archives 01, 02 and 05 compute integral cocycles spanning $H^1(\cdot; \mathbb{Q})$ in a spanning-tree gauge and convert a cocycle to normal coordinates by half-integral level sets of local vertex heights. The constructions agree (seven coordinates per tetrahedron, matching equations checked, Euler characteristic without disc expansion). All are honest that this is not a saturated $\mathbb{Z}$ -basis, not a connected surface, and not a cut. The four potential modules implement the same base- $(g+1)$ lemma and differ only in interface.

3.4 The grid search of archive 01

This is the one place where the review found a gap in an argument. The recognizer’s negative certificate is a set of grids closed under all implemented moves; soundness needs the implemented move set to be exactly a set for which monotonic simplification holds. The report states that “rows sharing an endpoint are not accepted by this implementation’s exchange rule” and cites Dynnikov’s Proposition 6, but does not argue that Dynnikov’s theorem still holds when exchanges of two adjacent rows (or columns) with a common corner column are excluded. Depending on the reading of Dynnikov’s definition of interleaving, that exclusion may remove legal moves, in which case a closed set could exist for an unknot. We could not settle this from the sources at hand, so we tested it: Section 4.3 compares the search verdict with Khovanov homology on every grid of size ≤ 6 and on random larger grids, and found no disagreement. The gap is in the proof, not (as far as tested) in the answers.

Everything else in archive 01 checked out: the move implementation, the Booth canonicalization, the independent verifier, and the exhaustive counts ($g!(g-1)!/2$ connected grids per size).

4 Cross-validation

None of the archives compared itself with another; each ran its own tests. We wrote three scripts (in `synthesis/data/`) that load the packages side by side.

4.1 Five Khovanov implementations on shared inputs

The same braid words and Knot Atlas PD codes were passed to all five packages (after converting label conventions). Reduced $\mathbb{F}_2$ ranks:

Input	02	03	04	05	06	agree
unknot circle	1	1	1	1	1	yes
trefoil	3	3	3	3	3	yes
figure-eight	5	5	5	5	5	yes
T(2,5)	5	5	5	5	5	yes
T(2,7)	7	7	7	7	7	yes
T(3,4)	5	5	5	5	5	yes
T(3,5) det 1	7	7	7	7	7	yes
unknot s1 s2	1	1	1	1	1	yes
unknot s1s2s3s4	1	1	1	1	1	yes
conjugated unknot	1	1	1	1	1	yes
hard unknot 05	1	1	1	1	1	yes
KT 11n42 (Atlas PD)	33	33	33	33	33	yes
Conway 11n34 (Atlas PD)	33	33	33	33	33	yes
10_124 (Atlas PD)	7	7	7	7	7	yes
30 random braid closures (3–4 strands, 6–10 letters)	ranks 1–13					30/30

All 44 inputs agree, including the two Alexander-polynomial-one 11-crossing knots and 30 random closures whose ranks range from 1 to 13.

4.2 Six pattern testers on random spherical patterns

Random perfect matchings of the darts of V trivalent vertices were kept when every component's rotation system is spherical, converted to the six input formats, and classified by all six testers (400 distinct systems per vertex count) together with six named patterns:

Vertices	spherical systems tested	random matchings drawn	essential	disagreements
2	400	883	102	0
4	400	737	12	0
6	400	1179	0	0
8	400	2603	0	0
10	400	6144	0	0

Pattern	verdict	all six agree
theta	essential	yes
tetrahedron	essential	yes
cube	essential	yes
prism	inessential	yes
dodecahedron	essential	yes
dumbbell	inessential	yes

No disagreement among 2006 patterns and three different criteria. (Random matchings on many vertices are almost always inessential because short bonds are common; the named cube and dodecahedron are the essential large examples.)

4.3 Grid search versus Khovanov homology

We wrote an independent grid-to-PD converter (traversal of the grid, crossing signs from the directions of the vertical and horizontal strands) and compared archive 01's search (determinant filter disabled) with archive 04's Khovanov rank:

Grid size	sample	diagrams	unknots	knots	disagreements
2	exhaustive (all shift orbits)	1	1	0	0
3	exhaustive (all shift orbits)	2	2	0	0
4	exhaustive (all shift orbits)	6	6	0	0
5	exhaustive (all shift orbits)	64	62	2	0
6	exhaustive (all shift orbits)	1212	1185	27	0
7	random, ≤ 14 crossings	300	286	14	0
8	random, ≤ 14 crossings	200	188	12	0
9	random, ≤ 14 crossings	100	82	18	0

The knotted orbits of size 5 are the two trefoils (grid number 5) and those of size 6 are 26 trefoil presentations and one figure-eight, exactly as the grid-number theory predicts. No disagreement was found, which supports but does not prove the completeness of the restricted move set. The sample stops at size 9 because the search itself becomes expensive there (one size-9 grid needed 900 863 states, and the 100 grids took 1.9 hours), while the Khovanov side stayed below 14 crossings by construction.

4.4 What the cross-validation does and does not show

Agreement of five independently written cubes is strong evidence that all five compute the same invariant, and agreement with the published ranks (trefoil 3, figure-eight 5, $T(3,5)$ 7, $K11n34$ and $K11n42$ 33) ties that invariant to reduced Khovanov homology. It says nothing about the quasi-polynomial question, and all runs are on diagrams of at most 14 crossings because the cubes cannot go further.

5 What is missing for $n^{O(\log n)}$

Consolidating the six audits with our own reading of the sources, an implementation with a proved $n^{O(\log n)}$ bound would require the following, none of which exists in any archive or, to our knowledge, in print.

1. **A representation.** A layered handle structure of the knot exterior with a Morse function, boundary-pattern labels, the ambient embedding in S^3 (used by the generalised Seifert construction of pp. 75–78), compressed normal-surface data, and regluing maps, with a proved bound on the bit size of all of this after arbitrarily many operations. The archives’ normal-coordinate modules are local ingredients only.
2. **Bounded multi-surfaces.** An algorithm producing independent relative homology classes with embedded representatives whose *pattern-sensitive* complexity is $O(n^2)$, and a proof that compression and pattern-compression preserve the invariants that allow later surfaces to be kept. A rational H^1 basis is not this.
3. **Compressed cutting.** Cutting, component extraction, Euler characteristic, orientation and pattern transport on compressed data without expanding parallel discs, with provenance sufficient to lift a violating dual cycle in a terminal ball to a disc in the correct earlier stage. Agol–Hass–Thurston orbit counting is a tool, not this procedure.
4. **Cheeger regions and weak reduction.** Recognition of the Heegaard–Morse hypotheses, construction of the modified generalised splitting, Haken’s lemma and weak reduction, and a global progress measure bounding restarts. The inequality $3g(\partial M_i) \leq \min\{g(H_i), g(H) - g(H_i)\}$ is a test on three integers, not the operation.
5. **The depth bound.** A proof that the executable process, after every rebuild and restart, has hierarchy length $O(\log n)$ (the talk) or $O((\log n)^2)$ (the Oxford version cited by archive 06), together with polynomial digit bounds. The preprint’s Section 10 gives a linear bound in the initial q -complexity, and its Section 9 explicitly declines to bound g and L .
6. **The cost accounting.** Only after all of the above does the conditional lemma proved in every archive, $L(g+1)^L$ iterations times a polynomial cost per iteration times polynomially many restarts, become a running-time theorem: $\log((g+1)^L) = O((\log n)^2)$.

The archives also list, and we agree with, the substitutions that do not work: an exponential search with a time cap is not a total algorithm; a polynomial iteration count does not bound work per iteration; expanding a compressed normal surface loses the compression; $b_1 = 0$ is not ball recognition; a Jones or Alexander test does not detect the unknot.

6 A faster exact recognizer: **fastunknot 0.1**

6.1 Design

Since the geometric route cannot be completed from the available sources, the second task was addressed by making the *exact* route as fast as we could while keeping every verdict provable. The package **fast/fastunknot** (standard library only, Python ≥ 3.10) runs:

1. validation (PD codes, braid closures, or grid diagrams in the archive-01 convention, converted by the traversal of Section 4.3);
2. crossing-decreasing Reidemeister I/II moves, as in archive 05;
3. the descending-diagram test of archive 02;
4. the *Alexander polynomial*, computed exactly by fraction-free Bareiss elimination over $\mathbb{Z}[t]$ on a first minor of the Fox matrix of the Wirtinger presentation (rows $(1-t)x_o + tx_u - x_v$ at positive and $(1-t)x_o - x_u + tx_v$ at negative crossings, with signs from the traversal); $\Delta \neq 1$ certifies knottedness and strictly dominates the determinant filter of the archives ($T(3,5)$ and all torus knots are now rejected in polynomial time; the Kinoshita–Terasaka and Conway knots are not);
5. reduced Khovanov rank over $\mathbb{F}_2$ by *scanning*.

6.2 The scanning computation

This is Bar-Natan’s local algorithm [4] specialised to $\mathbb{F}_2$ and implemented directly in terms of the arc-algebra description of morphisms. The crossings are processed one at a time in a greedy order chosen (from up to twelve starting crossings) to keep the boundary of the processed region small. After k crossings the partial complex has:

- objects: perfect matchings of the boundary edges, each with a homological degree; closed circles are delooped, one object with a circle becoming two objects (labels 1 and x);
- morphisms $a \rightarrow b$: $\mathbb{F}_2$ -combinations of dot-subsets of the circles of $a \cup \bar{b}$, i.e. elements of $A^{\otimes \text{circles}}$ with $A = \mathbb{F}_2[x]/(x^2)$.

Composition is the two-dimensional TQFT of the glued surface: a component of genus ≥ 1 or with two dots is zero (because the handle operator $m \circ \Delta = 2x$ and x^2 vanish), a closed component is 1 exactly when it carries one dot, and a planar component with β boundary circles and $d \in \{0, 1\}$ dots is $\Delta^{(\beta-1)}(x^d)$, which over $\mathbb{F}_2$ is $x \otimes \cdots \otimes x$ for $d = 1$ and the sum over one undotted position for $d = 0$. Delooping is the same computation with a cup on the source circle (dotted for label x) and a cap on the target circle (dotted to extract the coefficient of 1, since the dual basis of $\{1, x\}$ under the counit is $\{\varepsilon(x \cdot), \varepsilon\}$). After every crossing all invertible entries are cancelled: an entry $b \rightarrow c$ between equal matchings is a unit of the local ring $\text{End}(a) \cong A^{\otimes k}$ iff its expansion contains the undotted term, its inverse is the finite geometric series of the nilpotent part, and Bar-Natan’s cancellation lemma replaces ε by $\varepsilon - \gamma\varphi^{-1}\delta$. When the last crossing is added the boundary is empty, every morphism is a scalar, so a minimal complex has zero differential and its number of objects is the unreduced total rank; the reduced rank is half of it (archive 04’s two-copies lemma) and equals 1 exactly for the unknot.

Proposition 6.1 (Correctness). *On a validated one-component diagram, **fastunknot** returns **UNKNOT** iff the knot is trivial, unless a user-selected ceiling stops it, in which case it returns **UNKNOWN**.*

Proof. Steps 2–3 are isotopies and a sufficient triviality criterion; step 4 uses $\Delta_{\text{unknot}} = 1$. Step 5 computes the homology of the Khovanov complex: the tensor product of crossing complexes in Bar-Natan’s category, delooping, and Gaussian elimination are chain homotopy equivalences, and the closed complex is the standard one. The decision rule is the detection theorem with the universal-coefficient argument of the archives. $\square$

Proposition 6.2 (Cost). *Steps 1–4 are polynomial in n . Step 5 is $2^{O(n)}$ in the worst case.*

Proof. The Alexander determinant is $O(n^3)$ polynomial operations of degree $\leq n$ with polynomially bounded coefficients. In step 5 the objects before elimination after crossing k number at most $2 \cdot 2^c$ times the objects after crossing $k - 1$, where $c \leq 2$ closed circles can appear, so the whole

computation is dominated by the cube; compositions cost polynomial time in the boundary size. $\square$

The interesting quantity is the size after elimination. Empirically it stays close to (number of crossingless matchings of the boundary) $\times$ (a multiplicity that grows with the tangle’s homology), i.e. roughly $2^{O(w)} \cdot \text{poly}$ for scan width w . **We have no proof of this**, and there cannot be one in general for the multiplicity: the reduced rank of $\#_k K11n34$ is 33^k with Alexander polynomial 1, so any method that outputs the full homology is exponential on such inputs, whatever the scan width. A subexponential unknot recognizer along these lines would have to detect “rank > 1 ” without finishing the computation; we do not know how.

6.3 Validation

The package has 19 unit tests (polynomial arithmetic; Alexander polynomials of the trefoil, figure-eight, 5_2 , $T(3, 5)$, $\Delta = 1$ for the two 11-crossing knots and two hard unknots; symmetry $\Delta(t) = \Delta(t^{-1})$, $|\Delta(1)| = 1$, mirror invariance and odd determinant on 40 random braids; Reidemeister moves; the local endomorphism ring; $d^2 = 0$ after every crossing; independence of the processing order; resource-limit semantics; known ranks; an independent dense reduced-cube computation on 25 random braids; the command line). Beyond the tests, the scanner agrees with archive 04’s cube on 120 random braid closures (3–5 strands, 6–13 letters, with random processing orders), and reproduces the Atlas ranks 33, 33 and 7.

6.4 Measurements

Timings are on the same Windows machine as everything else in this report; they illustrate the dependence on scan width and are not complexity evidence.

Family	n	reduced rank	max. boundary	objects before	objects after	seconds
unknot sigma_1..sigma_n	8	1	2	6	2	0.001
unknot sigma_1..sigma_n	16	1	2	6	2	0.004
unknot sigma_1..sigma_n	32	1	2	6	2	0.013
unknot sigma_1..sigma_n	64	1	2	6	2	0.036
unknot sigma_1..sigma_n	128	1	2	6	2	0.133
unknot sigma_1..sigma_n	256	1	2	6	2	0.475
T(2,n)	5	5	4	30	10	0.003
T(2,n)	11	11	4	66	22	0.011
T(2,n)	21	21	4	126	42	0.043
T(2,n)	41	41	4	246	82	0.153
T(3,n)	8	5	6	42	10	0.010
T(3,n)	10	7	6	54	14	0.022
T(3,n)	14	9	6	78	18	0.032
T(3,n)	16	11	6	90	22	0.045
T(3,n)	20	13	6	114	26	0.092
T(3,n)	22	15	6	126	30	0.089
random 3-braid	20	75	6	762	150	0.217
random 3-braid	40	321	6	3354	642	1.035
random 4-braid	15	5	6	30	10	0.015
random 4-braid	21	25	8	438	146	0.576
random 4-braid	31	45	8	604	216	0.947
random 4-braid	41	109	8	654	218	2.683
random 5-braid	24	13	6	109	42	0.099
random 5-braid	36	timeout	timeout	timeout	timeout	600.147
random 6-braid	31	85	8	548	216	1.194

Example	n	verdict	deciding step	seconds
conway.json	11	KNOTTED	reduced-khovanov-F2-scan	0.0528
figure_eight.json	4	KNOTTED	alexander-polynomial	0.0001
grid_determinant_one_knot.json	10	KNOTTED	alexander-polynomial	0.0010
grid_scrambled_unknot.json	6	UNKNOT	reidemeister-reduction	0.0004
hard_unknot_8.json	8	UNKNOT	reduced-khovanov-F2-scan	0.0022
kinoshita_terasaka.json	11	KNOTTED	reduced-khovanov-F2-scan	0.0593
torus_3_5.json	10	KNOTTED	alexander-polynomial	0.0009
trefoil.json	3	KNOTTED	alexander-polynomial	0.0001
unknot.json	0	UNKNOT	reidemeister-reduction	0.0000
unknot_braid40.json	40	UNKNOT	reidemeister-reduction	0.0090

One case timed out: a random 36-letter closure on five strands exceeded the 600-second budget, while a 31-letter closure on six strands took about a second. When this section was first written we attributed the difference to the size of the minimal complexes of the intermediate tangles. **That explanation was wrong.** Section 7 shows that the minimal complexes of this input never exceed 5898 objects; the time went into fill-in caused by cancelling pivots in last-in-first-out order, and with min-fill pivots the same scan takes under a second.

Two comparisons with the archives' cubes on identical inputs: the Kinoshita–Terasaka knot (11 crossings, 2048 states, 11 559 generators) takes 0.24–0.31 s in the cubes and 0.07 s in the scan with a boundary of six edges and at most 66 objects after elimination; the unknot $\widehat{\sigma_1 \cdots \sigma_n}$, archive 03's explicit exponential family, needs 2^n states in every cube and is decided by the scan in 0.011 s for $n = 80$ with two objects after every elimination.

6.5 Limits of the new package

It is a reference implementation in pure Python; a compiled implementation of the same algorithm (as in Bar-Natan's or the KnotJob programs) would be far faster. Its scan-order heuristic is greedy; for diagrams whose Tait graph is grid-like the width is $\Theta(\sqrt{n})$ at best and the minimal complexes can be large. It does not produce a spanning disc or a Reidemeister sequence. And, to repeat, it is not quasi-polynomial.

7 Nine acceleration proposals

7.1 What was delivered

After version 0.1 of `fastunknot` (Section 6) was written, nine further runs were each asked to accelerate it. Their archives were extracted into `proposals/01–09` (wrapper directories removed, ZIP files deleted, archive order recorded in `proposals/ORDER.txt`). Every proposal is a complete replacement package with its own report, tests, an unchanged copy of the 0.1 baseline and same-machine benchmarks, and every one of them states that the general worst case remains exponential. As with the six original archives, the independent runs converged on the same ideas, which is evidence that the ideas are the natural ones.

7.2 The ideas

1. **Units are involutions.** In $\text{End}(m) \cong \mathbb{F}_2[x_1, \dots, x_k]/(x_i^2)$ a unit is $1 + \nu$ with ν in the maximal ideal, and $\nu^2 = 0$ in characteristic two (squares of monomials vanish, cross terms occur twice). So $(1 + \nu)^{-1} = 1 + \nu$ and the geometric series of 0.1 is unnecessary. All nine.
2. **Min-fill pivots.** Version 0.1 cancelled invertible entries in last-in-first-out order. Choosing the pivot $b \rightarrow c$ that minimizes the Markowitz count $(\text{out}(b) - 1)(\text{in}(c) - 1)$ through a lazy heap limits fill-in [6]. Seven of nine (not 05, 06).

3. **Cheaper cobordism algebra.** The topology of a gluing (which discs merge, Euler characteristics, boundary circles) does not depend on the dots, so it can be computed once per triple of matchings and reused; identity and square shortcuts; memoized results. All nine; proposal 03 goes furthest and packs a morphism into one integer whose bit number is a dot mask.
4. $O(n \log n)$ **scan order** and $O(n)$ **descending test.** The greedy score of a crossing changes only when a neighbour is processed, so a lazy heap reproduces the 0.1 order exactly; the descending test becomes a sweep over a difference array of bad cyclic intervals. All nine.
5. **Modular Alexander test.** Evaluate the first Alexander minor in a prime field; for the unknot it is $\pm t^k$, $0 \leq k < n$, so any other value certifies knottedness in $O(n^3)$ field operations. Six of nine, three of them only at $t = -1$ (the determinant).
6. **Modular Jones test by a frontier scan.** Evaluate the Kauffman bracket at a fixed $A \in \mathbb{F}_p$ while scanning the diagram, adding the weights of partial states as soon as they induce the same boundary matching. A value different from $\delta(-A^3)^w$ certifies knottedness. All nine. This is the most valuable single idea for recognition: it rejects the Kinoshita–Terasaka and Conway knots, which have trivial Alexander polynomial, in about a millisecond, and its cost is *provably* $\text{Catalan}(w/2) \cdot \text{poly}(n)$ for scan width w , because states with equal matchings are merged; there is no multiplicity factor.
7. **Visible connected sums.** Two edges bordering the same two faces are a two-edge cut of the projection, so the diagram is a connected sum of the two sides. Reduced Khovanov homology over a field is multiplicative, so ranks multiply and degree counts convolve; for recognition the summands are examined separately. Seven of nine.
8. **Crossing signs from both strands.** Version 0.1 assumed that a PD row starts at the incoming under-port (true for its own converters); seven proposals compute the sign from both incoming ports, which the Jones normalization needs.
9. **Incremental Reidemeister simplification** (02 only), **no cancellation on the last crossings followed by linear algebra** (06 only), **an exact integer bracket after the modular one** (05 only), and **witness replay commands** (six proposals).

7.3 Same-machine comparison

The harness `synthesis/data/proposals_bench.py` runs every engine on the same inputs, each measurement in a fresh process with a 120-second limit; timers exclude start-up, imports and input construction; medians of three runs (one run above five seconds). “scan” is the raw Khovanov scan with factorization disabled, “rank” the rank of a connected sum of eight Conway knots through each engine’s factored API when it has one, “recognize” the default pipeline. Milliseconds; 0.1 is the baseline, 0.2 the integrated version of Section 8, and Rust the port of Section 9.

Task	0.1	01	02	03	04	05	06	07	08	09	0.2	Rust
scan: conway	34.4	11.6	10.4	9.86	11.3	10.2	10.4	9.80	10.0	12.5	8.44	0.59
scan: kinoshita_terasaka	41.5	12.4	11.2	10.5	12.8	12.3	13.3	10.8	11.3	13.0	9.05	0.62
scan: T(3,11)	51.8	29.9	29.4	27.9	31.2	28.8	31.7	26.9	30.3	30.8	26.8	1.22
scan: braid3_40	611	103	115	94.0	108	133	140	143	164	201	84.1	6.88
scan: braid4_41	984	263	274	212	350	520	321	273	268	284	160	10.2
scan: braid6_31	495	96.7	108	88.2	115	95.1	93.6	94.6	104	114	67.7	5.25
scan: braid5_36	>120 s	1064	1208	887	1153	44995	7708	1080	1163	1491	691	68.1
scan: chain256	312	28.8	24.9	29.7	26.9	24.8	335	24.1	24.2	25.3	19.3	0.97
scan: conway_sum_2	1481	239	241	173	204	220	205	213	207	261	136	11.9
rank: conway_sum_8	>120 s	31.5	>120 s	20.2	31.4	>120 s	16.0	29.3	51.7	18.6	20.7	1.90
recognize: trefoil	0.13	0.19	0.19	0.13	1.67	0.15	0.15	0.28	0.16	0.15	0.27	0.008
recognize: torus_3_5	0.98	0.34	1.03	0.32	2.24	1.06	0.80	1.04	0.32	0.66	0.56	0.027
recognize: hard_unknot_8	2.84	5.05	2.40	4.02	3.86	2.45	3.07	2.18	2.16	3.03	2.78	0.056
recognize: unknot_braid40	11.2	8.00	0.41	7.82	8.31	7.64	7.50	7.93	8.64	8.42	0.17	0.005
recognize: conway	58.8	1.53	0.89	1.79	4.01	3.50	4.05	1.57	3.95	1.20	1.27	0.055
recognize: kinoshita_terasaka	65.6	2.12	1.02	1.49	3.43	1.55	3.31	1.08	1.81	0.72	1.25	0.030
recognize: conway_sum_2	1817	3.54	2.05	3.47	2.75	2.32	12.6	2.07	2.77	2.17	1.56	0.059
recognize: conway_sum_3	58090	6.77	2.96	5.15	3.60	3.43	35.3	2.10	2.78	2.51	2.47	0.099
recognize: T(3,61)	4747	3.71	12.5	5780	13.8	19.0	5811	22.3	5.91	8.80	8.84	0.19
recognize: braid5_36	9.89	2.53	0.69	2.38	8.63	2.63	11.2	6.20	2.40	2.37	0.90	0.023

All eleven Python engines and the Rust binary return identical ranks and verdicts on every input. Observations:

- The stress case **braid5_36**, which 0.1 abandoned after 600 seconds, takes about a second wherever min-fill pivots are used, 45 seconds in proposal 05 (LIFO kept) and 7.7 seconds in proposal 06 (LIFO plus the tail trick). Its reduced rank is 2949. **This corrects our own 0.1 analysis:** we had attributed the timeout to large minimal complexes of the intermediate tangles. The minimal complexes are small (at most 5898 objects); what exploded was fill-in caused by the pivot order.
- Proposal 03 has the fastest raw scanner, which is why its representation was adopted.
- On $T(3,61)$ proposals 03 and 06 take 5.8 seconds. Proposal 06 runs the symbolic Alexander polynomial before anything modular. Proposal 03 is more instructive: it evaluates the minor at $t = 2$ modulo $2^{31} - 1$, where 2 has multiplicative order 31, so the set of “units” $\{\pm 2^k\}$ has only 62 elements and is closed under the cyclotomic arithmetic of torus knots; $\Delta_{T(3,61)}(2) \equiv -2^{29}$ and the filter misses. Our own first implementation had the same structural weakness (2 has order 61 modulo $2^{61} - 1$) even though it happened to fire. Version 0.2 and the Rust port evaluate at fixed generic field elements instead, for both the Alexander and the Jones test. The filters are one-sided, so this was never a soundness issue, only a lost speedup.
- Tiny inputs get slower: the trefoil goes from 0.1 ms to 0.2–0.3 ms in most engines, because more stages run before one fires.

8 fastunknot 0.2

8.1 What was adopted

Version 0.2 (**fast/**) integrates every proposed idea that produced a measurable speedup, re-implemented in the structure of the existing package; the 0.1 scanner is kept unchanged as **scan_reference** so that each idea can be switched off and so that the new code has an independent oracle.

The pipeline is now: validation; incremental R1/R2 reduction; linear descending test; visible connected-sum factorization (summands examined smallest first, the first knotted one decides, and a sum is trivial exactly when all summands are); then per summand the Alexander minor at $t = -1$ and at a generic field element modulo $p = 2^{61} - 1$, the Kauffman bracket at a generic A modulo p (skipped when it exceeds 4096 frontier states), the exact Alexander polynomial over

$\mathbb{Z}[t]$, and finally the Khovanov scan with bit-packed morphisms, compiled plans and min-fill pivots. Every filter is one-sided: only a mismatch with the unknot’s value gives a verdict, and that verdict is KNOTTED.

Not adopted: the exact integer bracket (equality with the unknot’s value is inconclusive whether it is computed modulo p or over $\mathbb{Z}$, so it cannot change a verdict, and it showed no measurable effect); replay commands (useful, but not a speedup); tails longer than one crossing (slower, see below).

8.2 Ablation

`fast/ablation.py` measures each idea in isolation, again in fresh processes (limit 300 s). First the scanner, in milliseconds:

Scanner configuration	conway	braid3_40	braid4_41	braid6_31	braid5_36
0.1 reference scanner	42.9	1115	1264	612	>300 s
sets, LIFO, series inverse	44.1	1044	1206	618	>300 s
sets, LIFO, self-inverse	43.0	703	1113	618	>300 s
sets, min-fill	43.0	594	845	553	10376
bits, LIFO	11.4	106	203	73.4	35622
bits, min-fill (default)	11.1	103	224	78.6	670
bits, min-fill, tail 1	10.3	97.0	232	81.8	566
bits, min-fill, tail 2	13.7	105	225	84.6	815

Then the polynomial-time stages and the filters:

Task	new (ms)	old (ms)	ratio
greedy order, 2048 crossings	heap $O(n \log n)$: 32.6	0.1 rescoring $O(n^2)$: 26685	818×
descending test, 2048 crossings	linear sweep: 3.72	0.1 quadratic: 314	84×
R1/R2 reduction, 2048 crossings	incremental: 3.77	0.1 rebuild per move: 28089	7458×
$T(3,61)$ rejection, 122 crossings	modular minor: 8.07	0.1 exact $\mathbb{Z}[t]$: 5829	722×
recognize Conway	with Jones filter: 0.81	without Jones filter: 10.0	12×
recognize Conway#Conway#Conway, no Jones	with factorization: 12.6	without factorization: 6781	539×
rank of Conway#Conway#Conway	factored: 20.7	one scan: 6972	337×

What the ablation shows:

- **The bit-packed algebra** is the across-the-board gain: 4 to 10 times on every scan, with either pivot rule.
- **Min-fill pivots** matter on exactly one kind of input, and there they are decisive. On the stress case the bit-packed scanner needs 35.6 s with LIFO pivots and 0.67 s with min-fill; the set-based scanner goes from more than 300 s to 10.4 s. On the other four inputs the pivot rule is neutral to slightly negative (the heap costs a few percent). The final complex is the same in all configurations, so the difference is fill-in during cancellation, not the size of the homology.
- **Self-inverse units** save between nothing and 30%, depending on how many cancellations involve a non-identity unit.
- **The tail trick** is mixed: one crossing without cancellation is 16% faster on the stress case and 6% faster on one other input, a few percent slower on the rest; two crossings are slower almost everywhere because the uncanceled complex quadruples. It is kept as an option (**tail**) with default zero.
- **Heap ordering, incremental reduction, modular Alexander test**: three to four orders of magnitude at 2048 crossings and at $T(3, 61)$. These are asymptotic improvements of polynomial

stages: the old code spent 27 s on ordering and 28 s on R1 moves for a diagram that reduces to nothing.

- **The linear descending test** needs an adversarial input to show (a 2048-crossing chain with one crossing flipped: 3.7 ms against 314 ms); on typical inputs the old test exits early and the two are equal.
- **The Jones filter** turns recognition of the Conway knot from a 10 ms Khovanov scan into a 0.8 ms bracket evaluation, and **factorization** turns a 6.8 s scan of three Conway summands into 12.6 ms even with the Jones filter disabled.

8.3 Complexity: what changed and what did not

The polynomial stages improved asymptotically: scan ordering from $O(n^2)$ to $O(n \log n)$ per start, the descending test from $O(n^2)$ to $O(n)$, R1/R2 reduction from one $O(n)$ revalidation per move to $O(1)$ per move, and the Alexander rejection from fraction-free elimination over $\mathbb{Z}[t]$ to $O(n^3)$ field operations.

Two statements with an exponential flavour are now *proved* rather than observed:

Proposition 8.1. *The Jones filter costs $O(n \cdot \text{Catalan}(w/2) \cdot \text{poly}(w))$ field operations for a scan of width w .*

Proof. After each crossing the state is a map from crossingless matchings of at most w boundary points to $\mathbb{F}_p$; each entry spawns two successors. $\square$

Proposition 8.2. *If the visible factors of the reduced diagram have $n_1, \dots, n_k$ crossings, recognition and the factored rank cost $\text{poly}(n) + \sum_i 2^{O(n_i)}$. In particular the cost is polynomial when every $n_i = O(\log n)$ and quasi-polynomial when every $n_i = O(\log^2 n)$.*

This is a statement about a restricted class of diagrams (several proposals make it too), not about unknot recognition: a composite knot can be drawn without any two-edge cut, and prime knots gain nothing. The family $\#_k K11n34$, which Section 6 used to show that a scan cannot be subexponential if it must output the whole homology, is exactly the family that factorization now handles in polynomial time; the obstruction moves to diagrams in which the summands are hidden.

The general worst case of the exact fallback is unchanged, $2^{O(n)}$, and the $n^{O(\log n)}$ target of Section 5 is no closer: nothing here touches the hierarchy machinery.

8.4 Verification

The package now has 33 unit tests. The 14 new ones compare bit-packed with set-based composition on random planar matchings, check every unit of a three-circle endomorphism ring, run all scanner configurations against the 0.1 scanner, check that heap orders equal the 0.1 orders from every start, check sign and filter invariance under half-turns of PD rows, check that no filter fires on any unknot diagram and that every filter hit has reduced Khovanov rank above one, check 33^k for $k = 2, 3, 8$ Conway summands and equality of factored and unfactored degree counts, replay simplifier traces, and recompute the stress case (rank 2949, with and without the tail).

9 A Rust implementation

9.1 Design

`rust/` is a dependency-free crate (about 2500 lines) implementing the 0.2 pipeline except the exact Alexander polynomial over $\mathbb{Z}[t]$, which would need big integers and on which exactness never

depended. It was designed for Rust rather than transliterated:

- matchings are interned, so objects, plans and caches hold 32-bit ids;
- a morphism is a bitset over dot masks, a single inline `u64` when the Hom space has at most six circles (every scan of width at most twelve) and boxed words otherwise;
- composition and transfer plans are a few integer masks per surface component, evaluated with popcounts;
- memo tables use an Fx-style multiplicative hasher; measured against the standard SipHash it is worth 22% on the stress case (65 ms against 86 ms) and 18% on two Conway summands;
- objects live in flat vectors, the min-fill queue is a binary heap, and field arithmetic modulo $2^{61} - 1$ uses 128-bit products;
- factored ranks such as 33^{16} use a small base- 10^9 integer.

9.2 Verification

Four unit tests (known ranks under both pivot rules, tails 0–2 and mirrors; input validation; one-sidedness of the filters; the pipeline), and `rust/cross_check.py`, which compares ranks by cube degree in three configurations and verdicts in two with the Python package on random braid closures: 200 closures, no disagreement.

9.3 Profile

`rust/profile.py` records the binary’s own timers (medians of five runs inside one process, after parsing). The comparison columns of the table in Section 7 show the Rust binary against the Python engines on identical inputs. Larger random braid closures, raw scan, one run each with a 300-second budget:

Input	reduced rank	boundary	objects	total s	transfer s	eliminate s
random 3-braid, 80 crossings	5189	6	12240	0.235	0.101	0.122
random 3-braid, 160 crossings	budget of 300 s exhausted					
random 4-braid, 61 crossings	643	8	3310	0.113	0.055	0.055
random 4-braid, 81 crossings	budget of 300 s exhausted					
random 5-braid, 48 crossings	39	10	450	0.015	0.011	0.004
random 5-braid, 60 crossings	481	10	7398	4.663	0.525	4.108
random 6-braid, 43 crossings	177	10	530	0.023	0.014	0.008
random 6-braid, 55 crossings	1171	10	13294	1.484	0.434	1.021
random 7-braid, 48 crossings	29	10	1838	0.070	0.032	0.036
random 8-braid, 49 crossings	1709	12	3418	0.309	0.153	0.149

What the profile shows:

- On identical inputs the Rust scanner is 10 to 17 times faster than Python 0.2 (Conway 0.59 ms against 8.4 ms, the 40-crossing 3-braid 6.9 ms against 84 ms, the stress case 68 ms against 691 ms) and recognition takes between 5 and 200 microseconds. Against version 0.1 the stress case went from more than 600 s to 0.068 s.
- Time splits roughly evenly between adding crossings (gluing, delooping, transfer plans) and cancellation, except when fill-in dominates: with LIFO pivots the stress case takes 2.1 s in Rust, 2.03 s of it in cancellation, against 0.068 s with min-fill. The pivot rule matters as much in Rust as in Python; on the other inputs it is again neutral.
- One uncanceled tail crossing saves 5 to 20% in Rust as well.
- **The exponential wall is still there.** A random 3-strand closure with 80 crossings has reduced rank 5189 and takes 0.23 s; with 160 crossings the scan exhausted 300 s. A 4-strand closure with 61 crossings (rank 643) takes 0.11 s; with 81 crossings it exhausted 300 s. The scan

width is 6 or 8 throughout, so this is the multiplicity factor of Section 6 and not the boundary: the rank of the homology itself grows exponentially with the length of a random braid word, and any algorithm that writes the homology down inherits that. (Both runs overshoot their cooperative deadline by 40 to 70 seconds, because the deadline is checked between cancellations.) For *recognition* these inputs are easy, since the Alexander test rejects them in microseconds; the hard inputs for recognition are diagrams of the unknot and of knots with trivial Alexander and Jones values at our evaluation points, where the scan must run to completion.

9.4 Where the three versions stand

Task	Python 0.1	Python 0.2	Rust
Scan, stress closure (36 crossings, rank 2949)	> 600 s	0.69 s	0.068 s
Scan, Conway knot	34 ms	8.4 ms	0.59 ms
Recognize Conway knot	59 ms	1.3 ms	0.055 ms
Recognize Conway#Conway#Conway	58 s	2.5 ms	0.099 ms
Recognize $T(3, 61)$	4.7 s	8.8 ms	0.19 ms
Rank of eight Conway summands (33^8)	> 120 s	21 ms	1.9 ms
Recognize the 8-crossing hard unknot	2.8 ms	2.8 ms	0.056 ms

The last row is the honest one for the original question: on an unknot diagram that no filter can decide, the accelerations of the Python version change nothing, because the Khovanov scan still has to run; only the constant factor of the Rust port helps. (This describes 0.1 against 0.2. The later work of Section 10 does speed up the scan itself, and this case with it, by a factor of 1.4.)

10 A third round: constant factors in Python, and how to measure them

After version 0.2 the Rust work was paused and the Python package was optimized further, with no change to the mathematics, the verdicts or the asymptotic behaviour. This section records what was gained, what was tried and rejected, and an episode about measurement that changed how every number below was obtained. All verdicts and ranks are exact and identical to those of 0.2 (the deciding step can differ: the Reidemeister III help below settles some unknots before the scan): 41 unit tests pass, and `crosscheck.py` compares the ranks by homological degree of the new default scanner with the generic 0.2 scanner and the 0.1 set algebra on random braid closures (400 closures with 2 to 5 strands, no mismatch).

10.1 What changed

A profile of 0.2 on the stress closure showed that the cobordism algebra was no longer the bottleneck: of 1.6 s under the profiler, the scanner’s own bookkeeping (method calls per differential entry, tuple-keyed dictionaries, hashing of nested frozensets, a heap of pivot candidates) took most of it.

- **A scanner written for CPython (`scan_fast.py`).** Objects of a stage are numbered $0, \dots, N - 1$ and live in lists; the objects created from one old object form a contiguous block, so a delooping label is an offset; matchings are interned as integers, so every cache key is a tuple of small integers; transfers of an edge are cached per (source matching, target matching, value) for both smoothings at once; cancellation is one inlined loop. The generic scanner of 0.2 is kept for the ablation configurations.
- **A bucket queue for pivots.** On the stress closure 114 197 unit entries are candidates but only 16 884 are used; the rest die when a neighbour is cancelled. Markowitz costs are tiny, so one list per small cost plus a heap for the rare large ones replaces the heap.

- **Integer planar geometry** (`planar.py`). Circles of $a \cup \bar{b}$ are found by walking partners alternately; a crossing is glued by following the at most four strands through it; a transfer plan is compiled only over the crossing and the circles it touches, the other circles being renumbered; and since monomials of different components live on disjoint circles, multiplying a bit-packed sum by a monomial c is the shift $\ll c$.
- **Outside the scanner.** The modular Alexander filter builds the matrix by sparse rows (at most three nonzero entries per row) and evaluates each distinct entry once; the boundary profile of a greedy scan order is tracked inside the greedy in $O(1)$ per step and a losing start is abandoned early; the Jones frontier scan uses the interned matchings and the strand-following gluing. Each was checked to be output-preserving against the previous implementation on 500 to 600 random diagrams, including diagrams with kinks.

10.2 Results

Same harness on both sides (`perf.py`: in one process, best of N , timers around the call only), so the columns are comparable with each other but *not* with the fresh-process medians of the previous section.

Task (milliseconds)	0.2	after the scanner work	ratio
Scan, stress closure (36 crossings, rank 2949)	575.4	216.8	2.65
Scan, 41-crossing 4-strand closure	135.3	51.1	2.65
Scan, 40-crossing 3-strand closure	71.0	26.2	2.71
Scan, 31-crossing 6-strand closure	59.3	21.4	2.77
Scan, Conway#Conway (unfactored)	114.9	41.7	2.75
Scan, $T(3, 11)$	22.8	8.7	2.61
Scan, Conway knot	7.7	3.0	2.54
Scan, chain of 256 kinks	15.9	10.9	1.47
Recognize the 8-crossing hard unknot	1.26	0.90	1.41
Recognize Conway knot, exact steps only	7.8	3.2	2.43
Total of the 14 cases of <code>perf.py</code>	1022.2	394.6	2.59

The right-hand column is a run whose control loop (see below) spread by 4.8%. The changes outside the scanner came later and were measured as paired ratios new/old with 95% confidence intervals, which do not need a quiet machine: recognition of $T(3, 61)$ 0.60 [0.53, 0.67] and then 0.69 [0.62, 0.72], together about 0.41; the chain of 256 kinks 0.78 [0.73, 0.82]; the Jones scan of the stress closure 0.67 [0.46, 0.72]; recognition of the Conway knot 0.92 [0.89, 0.93]. Four later sequential runs were all rejected by the control rule and are not used; they are flagged in `perf_log.json`.

A cache that began as a trade-off. The last change was at first not a free win; how it became nearly one is told in order. Every crossing brings new edge labels, so plans cached per stage are never reused across stages, although a long braid compiles the same few pictures at every crossing: at boundary size 6 there are five matchings, and about 35 transfer plans were compiled per crossing of $T(3, n)$. Transfer plans are now also cached under a label-independent key, the matchings and the crossing relabelled by rank. This is sound because the relabelling is monotone and circles are numbered by their minimal label whichever matching is walked first (tested on random matchings). The reuse is 80% of the plans for $T(3, 11)$, 94% for $T(3, 31)$, 99% for the chain of kinks, 38% and 19% for the random 3- and 4-strand closures, but 1% for the stress closure and none for the Conway knot. With the cache always on, three agreeing A/B runs: scan of $T(3, 11)$ 0.61 [0.59, 0.66], chain of kinks 0.78, 40-crossing 3-strand closure 0.97; *scan of the Conway knot* 1.06 [1.04, 1.07] *and recognition of the hard unknot* 1.07, *slower*; no claim on the large scans. Timing partial variants against the old code (A/A ratio 0.999) splits the cost into ranking the boundary at each stage (+1.5%), building and looking up the key (+3.0%) and storing (+1.3%).

Two explanations offered first were wrong: interned the shapes as integers, against the hashing of nested tuples, changed nothing, and the ratio was the same with the cyclic garbage collector disabled; `shape_cache=False` restores the previous speed exactly (1.003 [0.999, 1.007]).

A first attempt at an automatic switch failed, and I concluded too early that none was possible: an early hit count predicts the wrong way, since the inputs that profit get their first hit only after 35 to 150 lookups, while the stress closure gets 12 hits in its first 64 lookups and then none. A different predictor works. A plan can be reused only in a stage whose picture, the position of the four crossing slots among the boundary points up to monotone relabelling, has occurred before, and that is known from the scan order alone, before the scan. The number of such stages is 0 for the Conway knot, the Kinoshita–Terasaka knot and Conway#Conway, 1 of 36 for the stress closure, 4 of 31 and 11 of 41 for the 6- and 4-strand closures, 16 of 40 for the 3-strand closure and 16 of 22 for $T(3, 11)$. The default now turns the cache on when the diagram has at least 16 crossings and an eighth of its stages repeat. Transfer and composition *results* are shared across stages too, but only after the first reused plan of a scan, because before that no shared result can exist; without that gate they cost a further 2.6% on the small irregular scans. Against the code before any of this, with 301 interleaved pairs: scan of $T(3, 11)$ 0.505 [0.486, 0.508], Conway knot 1.008 [1.005, 1.010], hard unknot 1.007; on the corpus, $T(3, 11)$ 0.51, chain of kinks 0.81, and nothing slower.

A stage that was not needed. The benchmark corpus contained one input that no filter decides, the hard unknot with 8 crossings, and that hid the cost of a whole stage. A generator of scrambled unknot braids (`hard_unknots.py`: the closure of $v\sigma_1\cdots\sigma_{n-1}v^{-1}$ rewritten by random braid relations) gives diagrams of which about half survive the Reidemeister I/II reduction with 15 to 27 crossings and pass every filter. On 26 of them the exact Alexander polynomial over $\mathbb{Z}[t]$ took a median 24%, and up to 63%, of the recognition. That stage can only answer KNOTTED, and only for a polynomial that is not a unit, which the modular test has just failed to see at $t = -1$ and at a generic point of $\mathbb{F}_p$; and whatever it might still catch, the exact scan decides anyway, so verdicts cannot change. It now runs only when the modular test did not. Over 720 diagrams the two modes never disagree, and the exact stage decided nothing once the modular test had passed. Recognition of the 55 scan-decided scrambled unknots is faster on all 55: median ratio 0.81, range 0.53 to 0.93.

Reidemeister III, the one algorithmic gain of this round. The scan is exponential in what the reduction leaves, and the reduction used to stop wherever a Reidemeister III move was needed. On the dart structure of the reduction a triangular face is a 3-cycle of the face walk through three distinct crossings; it can be inverted if and only if some side passes over at both ends; and inverting it only reverses the order in which each of the three strands meets the other two, a relinking of at most twelve darts that keeps every crossing’s slots and the direction of both strands through it, hence its sign. When no I/II move is left, each such triangle is inverted on trial, kept if that creates a I/II move and undone otherwise (the move is an involution). The move was tested on its own before it was used: 150 random moves give valid spherical diagrams with the same Alexander polynomial, writhe and Khovanov ranks by degree, and a second move restores the structure exactly (300 of 300). On 1500 random closures the helped reduction never leaves more crossings than I/II alone, leaves no I/II move, replays from its trace, and preserves the Alexander polynomial and the Khovanov rank; 26% of the diagrams shrink further and 0.80 of the crossings remain. The 8-crossing “hard unknot” used throughout this report goes from 6 crossings to none, and so do all 120 scrambled unknots above, which is no surprise, their scrambling being made of such moves. Honest hard inputs had to be found by search: 117 closures among 40 000 random words that survive I, II, the III search and the Alexander filter, and are unknots by reduced Khovanov rank 1, with up to 21 crossings. Verdicts with and without the search agree on 900 diagrams.

Two lessons came with it. *Placement*: run together with the first reduction, the search cost 58% on $T(3, 61)$, 20% on a random 5-strand closure and 4 to 6% on the Conway knot, inputs that a filter decides a moment later. It now runs only after every filter has failed, immediately before the scan; then no slowdown can be claimed on those inputs, while recognition of the hard unknot takes 0.48 and of a scrambled unknot of 27 crossings 0.18 of the time. *Fewer crossings is not always less work*: one survivor was flagged 1.43 times slower, correctly, since the help takes it from 22 to 20 crossings and its scan work from 2081 to 3277. Over the 54 survivors where crossings are removed (3.4 on average) the scan work is 0.43 of what it was, cheaper on 44 and dearer on 10, with per-diagram ratios from 0.03 to 7.3; over all 117 it is 0.69.

Depth, and a budget. One move of lookahead left those 117 unknots untouched. Sequences of two moves, the second next to the first, reduce 97 of them to nothing, three moves 109 and four moves 110, without a violation at any depth. What costs is a search that fails: on $T(3, 61)$, full of triangles that lead nowhere, depths 1 to 4 take 2, 10, 44 and 189 ms. Useful sequences are found early, so the search watches its own cost and deepens only while fewer than a fixed number of trial moves per crossing have been made; budgets of 5, 15, 40 and none give the same 97, 109 and 110, and a budget of 5 caps the $T(3, 61)$ case at 10 ms. With the defaults (depth 4, budget 10) only 15 of 60 000 random closures are unknots that survive the reduction, the largest with 16 crossings.

Introspective selection and racing (two suggestions of the user). The first suggestion was to choose algorithms from a runtime profile, as introsort does. It works where a cheap signal separates the populations and switching is free, and three parts of the package are now of that kind: the shape cache chosen from the scan order, result sharing enabled by the first reused plan, and the budgeted deepening just described. For scan *orders* it fails with the rules at hand, in both forms tried. Restarting with another tie-break rule when a growing work budget is overrun was simulated exactly on the saved work of three rules on 81 diagrams: all 27 policies cost 1.4 to 4.8 times the default. Introsort can switch because heapsort carries a guarantee and recursion depth signals degeneracy; here no rule has a guarantee, expensive is not degenerate, and an abandoned scan is lost. Without restarts (the previous stage is never mutated, so tied candidates can each be tried from the same state and the smallest real complex kept) the cost with all trial work counted is 0.41 on the scrambled unknots but 1.44 to 1.52 on random closures and about 1.4 overall: the size of the complex now does not predict the cost of the choice later.

The second suggestion, racing two algorithms on separate cores, is where the same data does promise something, because by wall clock a race costs only the winner's time: the better of two rules needs 0.75 of the default on the expensive inputs, and 0.07, 0.24 and 0.30 on single ones. In Python a competitor has to be a process, and starting one costs 30 ms bare, 76 ms with the package imported and 111 ms for a complete scan of the trefoil from the command line, against scans that mostly finish within 300 ms. Following the user's instruction for that case, the race was built in the Rust version, where a thread costs microseconds, together with a port of the Reidemeister III search: up to three greedy orders (ties by crossing index, by longest wait, by most recent touch) are scanned on separate threads and the first to finish stops the others through a flag; the rank does not depend on the order, so the result is the same whoever wins. On 40 random closures of 34 to 55 crossings whose single scan takes 30 ms to 8 s, with the settings interleaved over five rounds, racing two orders needs 0.650 of the total time and racing three 0.689. The gain is insurance against the heavy tail and not uniform: per input the ratio runs from 0.05 to 1.68; 6.8 s become 0.34 s and 890 ms become 140 ms, but on the half of the inputs where the default order was the best anyway the race costs 10 to 60%. I first blamed the shared heap for that premium and was wrong: two or three separate processes running the same single scan at once slow each other down exactly as much as the racing threads (1.15 against 1.18, 2.11 against 2.06), so it is contention in the hardware of this machine and will differ elsewhere. The race is therefore an option (`-race 2`), not the default.

Dismissing a race in Python was, on reflection, a wrong comparison: the cost of starting a process

was set against the typical scan, while a race only matters on the heavy tail, where Python scans take seconds to minutes. The Python package therefore has it too, in the form the two suggestions combine to: the default order runs alone for a head start of one second, and only then are other greedy orders started as separate processes, which the scanner polls where it looks at the clock; the first to finish wins. On 26 random closures with single scans of 0.15 to 3.6 s the total is 0.910 of single with two orders and 0.925 with three. That total consists of two inputs on which a competitor wins in every round (3.22 s to 1.74 s, 3.55 s to 1.56 s) and of a cost of 10 to 20% on scans of 1 to 3 s that the default order wins. The other per-input ratios of that experiment are noise, and the data says so itself: 18 inputs finish inside the head start, where nothing is started and the ratio must be 1, yet they read 0.80 to 1.43 with three rounds each. Measured with 61 interleaved pairs, an armed race that starts nothing costs 1.010 [0.984, 1.041] against an A/A control of 1.042, that is, nothing.

Start-up, the largest gain for a command-line user. Everything above saves well under a millisecond on a small diagram, while a whole `python -m fastunknot recognize` process took 115 ms, 29.5 of them for the bare interpreter. Importing the package cost 56 ms: 25 for `dataclasses`, which loads `inspect`, `re`, `dis`, `ast` and `tokenize`, used for five simple records (now written out by hand with the same constructors, equality, hashing, immutability and `repr`); 9 for `subprocess` and `json`, needed only when a race starts; 5 for `typing`, used only in annotations that are never evaluated. Then `argparse`: 55 to 59 ms in Python 3.14, where it loads `_colorize`, `dataclasses` again and `shutil`. The options are now one table feeding a small direct parser for the well-formed common case and, on demand, the real `argparse` parser for everything else, so help and error messages are unchanged; tests check that the two agree wherever the fast one accepts. With 41 interleaved pairs of whole processes a run takes 0.52 of the time (0.510 to 0.541; 115.5 to 60.9 ms for the Conway knot) on every short invocation and 0.86 on the half-second stress scan. My first two such comparisons showed no difference and were invalid: `python -m` puts the current directory first on the module path and both arms ran from the working tree, which was thus compared with itself.

Profiling `recognize` by cumulative time, rather than the scanner by own time, then found four more things of the same unglamorous kind. The Jones filter paid for the bookkeeping of a shape cache it never uses (9% of a recognition of the Conway knot); the reduction rebuilt and revalidated the diagram even when no move applied (12%); the dart involution of an immutable diagram was recomputed seven times per recognition; and on the commonest input of all, a reducible closure that the Alexander filter decides, a quarter of the time went into validating the reduced diagram from scratch, although the moves can only break its topology, which is now checked directly on the darts (one traversal through every crossing twice, $n + 2$ faces; of 282 deliberately corrupted structures 273 are rejected and the other 9 also pass the full validation). Against the code before: recognition of the Conway knot 0.765 [0.733, 0.794], of $T(3, 61)$ 0.759, of `Conway#Conway#Conway` 0.731, and of the random 5-strand closure 0.865 and then 0.796 [0.772, 0.831], with status, deciding step, reduced size and rank identical on 700 diagrams and reduced diagrams and traces identical on 1500. The `tail` option was re-tested on the present scanner and stays at 0: one crossing of tail is a wash (0.94 to 1.01), two or three are worse, up to three times.

Two remarks. First, the closing sentence of the previous section, that on an unknot diagram no filter can decide the Python accelerations change nothing, was true of 0.1 against 0.2 and is no longer true: that case is now 1.4 times faster, because the scan itself is. Second, the gain is a constant factor: between 2.5 and 2.8 on every scan in the table except the chain of kinks (1.5), where fixed work per crossing dominates. Nothing here touches the exponential growth, and the Rust port remains several times faster still.

10.3 Measured and not adopted

All of these were decided with deterministic counters (compositions performed during cancellation, differential entries written while adding crossings), and the ranks were identical in every variant.

- *One queued pivot candidate per source object.* Fewer queue operations, but 42 197 instead of 29 347 compositions on the stress closure: the queued candidate goes stale. Rejected.
- *Other pivot rules.* First-in-first-out inside a cost bucket needs 64% more compositions on the stress closure; the cost $(\text{out} - 1) + (\text{in} - 1)$ is worse on all four inputs tried. The product cost with last-in-first-out stays.
- *Another score for scan orders.* Scoring greedy orders by $\sum 2^{|\partial|/2}$ selects the same start as the current rule (maximal, then total boundary size) on all four inputs.
- *Scoring all greedy starts instead of twelve.* On 40 random closures the order changed in 18: equal work in 8, less in 6, more in 4, with ratios from 0.29 to 6.69. The total, 0.952, is carried by three inputs and changes sign without one of them. Not adopted.
- *Cancelling saddle isomorphisms before writing their entries.* Between 74 and 83% of the entries written are incident to a cancelled pair when it is cancelled, but only 30 to 53% of the cancellations are between the two smoothings of one old object, the predictable kind, and they account for 24 to 36% of the entries. That bounds the gain near 15% on the stress closure before the cost of the extra code. Not pursued.
- *A fast path for single-entry transfers.* It would apply to 13% of the edges of the stress closure: too little to measure. Not written.

The experiments on scan orders have a positive content. The score is the weak point, not the number of candidates: on the stress closure the greedy starts 14 to 16 need 23 710 compositions and have boundary profile (8, 226), while start 0, with profile (8, 230), needs 895 182, that is 38 times more, with 2.6 times the peak size; 20 of the 36 starts do not finish in 4 s. A matchings-only simulation cannot replace the score, because the gap occurs at equal boundary size, where there are at most 14 matchings: it is all multiplicity, i.e. homology of the partial tangle, which is what the computation computes. A cheap predictor of that multiplicity would be worth more than everything in this section.

A larger experiment, on 81 diagrams (36 scrambled unknots of the kind described above and 45 random closures with 20 to 35 crossings), confirms this and closes one more door. The greedy rule breaks ties by crossing index, which is arbitrary. Breaking them towards the most recently touched crossing costs 1.09 and 1.05 of the present total work on the two families, and towards the longest-waiting crossing 1.52 and 1.57; per diagram the ratios run from 0.01 to 28, better on about as many inputs as worse, so the rules are incomparable rather than ranked. A portfolio does not rescue this. The best of the three rules on each diagram would need 0.31 of the work on the unknots and 0.74 overall, but running them interleaved until the first one finishes costs three times the winner: 0.94 on the unknots and 2.2 overall, because 8 of the 81 inputs carry 86% of the work and there the rules lie within a factor two of each other.

10.4 An episode about measurement

Midway, a benchmark run showed everything 40% slower, including code that had not been touched. No process of ours was running; the desktop's own load had changed. A fixed pure-Python control loop, added to the harness, went from 5.3 ms to 12 ms *inside one run*. Sequential before/after timing was therefore abandoned for decisions: `perf.py` now times the control next to every case and rejects a run whose control spreads by more than 10% (eight of the fifteen logged runs are rejected), and `ab.py` times the old and the new code alternately in one process.

The first criterion of `ab.py`, “the interquartile range of the paired ratios lies below 1”, was then caught making false claims in both directions. A change confined to order selection, which is 0.2 to 1.8% of the scans concerned and was measured on its own as 32 to 58% faster with identical

output, was reported as 14% slower on one input and 4% faster on another. The bound came from asking what share of the run time the change could touch at all. The tool now times the old code twice per round; on mid-sized scans this A/A control shows identical code differing by 20 to 30% between adjacent runs. A difference is claimed only if the order-statistic 95% confidence interval of the median ratio excludes 1 *and* the median lies outside the interquartile range of the A/A ratios. The one claim that had been published under the lenient criterion (7 to 13% from the refinements of the geometry on small scans, no claim on large ones) was re-tested between the two revisions under the strict criterion and holds: 0.87 to 0.92 with intervals inside $[0.87, 0.95]$, no claim on the large scans.

A smaller finding of the same kind: the new scanner at first overshot a time budget of 1 s by up to 0.24 s (the generic scanner: 0.01 s), because a single crossing was added without looking at the clock. With a check every 512 objects the overshoot is at most 0.03 s on the same runs.

11 A port to Bend 2, and what its parallelism delivered

Bend 2 (version 2.0.10) is a pure, total, affine functional language that compiles through C to native code and promises parallelism for free: a parallel let $\mathbf{a} \ \mathbf{b} = \mathbf{f}(\mathbf{x}) \ \mathbf{g}(\mathbf{y})$ forks, the type checker guarantees that the two calls share nothing, and the result cannot depend on the number of threads. The algorithm of **fastunknot** was ported to it (directory **bend/**) to find out what that promise is worth for this problem. Nothing in this section changes the complexity discussion: the scan is exponential in the boundary width, in every language.

11.1 Setting

Bend runs on Linux, macOS and WSL only, so all Bend work ran over SSH on an Ubuntu virtual machine with 6 virtual CPUs of an i7-3770. That CPU has no AVX2 and the distributed binary died with an illegal instruction; Bend was run from source under the baseline build of its JavaScript runtime instead. All times below are from that host, and the Python package was copied there for every comparison, so the two sides of a ratio always share the hardware. Timings are interleaved with an A/A control as in Section 10; the control’s ratio stayed within 0.96 to 1.05.

11.2 What was ported, and how it was checked

- the modular Alexander filter (Wirtinger matrix, determinant modulo 65521, because U32 is the only machine integer);
- the planar geometry, the dotted cobordism algebra over $\mathbb{F}_2$ and Bar-Natan’s scan with delooping and Gaussian elimination, that is **scan_fast.py** with, of its caches, only the transfer plans;
- a greedy scan order from eight starting crossings;
- a standalone recognizer that chains the three.

Not ported: diagram simplification, the Jones filter, factorization, and the cache of composition plans. Each layer was compared with its Python counterpart before the next was written: 200 Alexander minors, 1920 gluings and circle numberings, 2736 compositions and transfers, the Khovanov ranks of 160 random closures and of the timing inputs (up to rank 5898), and the verdicts of the recognizer on 60 diagrams with up to 40 crossings, 40 of them unknots, among the knots Conway’s and the Kinoshita–Terasaka knot: no mismatch anywhere. Eighteen laws about the control logic (for instance: of two scan orders with equal profile the left one wins, so the work does not depend on scheduling) are stated in **LAWS.bend** and proved in **PROOF.bend**; they were drafted by the AI as proposals, and they do not say that a rank is right, which Bend cannot express about U32 arithmetic.

11.3 What the fork delivers

Measured on kernels first: a deep, balanced, uniform recursion reaches 5.0 on 6 threads; 32 coarse leaves reach 2.1; every switch from sequential to parallel execution costs about 80 microseconds; forks inside the sparse elimination of the Alexander determinant made it 8 to 15 times *slower*; one fork over four independent determinants gained nothing. Tasks are dealt once and never moved, and there is no cancellation, so the race of scan orders, the best parallel gain of the Rust port (0.65 of the time in total), cannot be written.

In the scan, the transfer of all objects of a stage through a crossing is a map that shares nothing, and it was written as a fork over the tree of the array. Its first effect was negative: with six threads the scan took 1.28 to 1.85 s instead of 1.04 s. The cause was found in the runtime source and the emitted C, and then confirmed by measurement: generated code makes a non-tail call either by pushing a stack frame or, in “parallel mode”, by allocating the continuation as a task on the heap; a program runs in parallel mode wherever a fork is reachable, which for a program with one fork is everywhere. With the forking transfer *switched off* and six threads the scan took 1.85 s: a factor 1.8 for the mere presence of a fork. The remedy is to make every long sequential phase a task of its own, a fork next to a trivial second child, since a task whose function reaches no fork runs in the fast mode. Three natural ways of writing that child were silently compiled away (inlined, constant-folded, dropped as unused), leaving no fork; it has to be recursive, depend on run-time data, and its result has to be used. After that: 1.02 s on one thread, 0.81 s on six; the transfer alone became 2.7 times faster.

11.4 Where the time goes

version	braid closure, rank 218		stress closure, rank 5898	
	1 thread	6 threads	1 thread	6 threads
first working scan, first-found pivots	1.04 s	1.28–1.85 s	63 s	58 s
sequential phases as tasks	1.05 s	0.89 s	63 s	58 s
least-fill pivots in batches, work lists	0.65 s	0.50 s	7.0 s	4.9 s
one transfer plan per pair of matchings	0.36 s	0.39 s	4.07 s	3.9 s
composition plans once per source matching	0.32 s		3.59 s	
one alternative evaluated instead of all	0.24 s		2.65 s	
number of sources kept, not counted	0.23 s	0.25 s	2.20 s	1.99 s
Python package, same host	0.134 s		0.542 s	

The largest gain was algorithmic and sequential: cancelling only pivots whose fill-in is within a limit, and raising the limit to the least fill-in that was skipped, brought the stress closure from 58 to 5 s. A sampling profile of the emitted C then showed 43% of the time in reference counting: a value used more than once is shared and counted, and every step through a shared list bumps and drops counts. Matchings are shared by design, and the linear lookups in them were most of those steps; a differential profile put the transfers at 48 to 61% of the single-thread time.

Python avoids that work by interning matchings and caching every plan (2817 plans serve 251 406 transferred entries on the stress closure). In Bend a cache is a value every call must thread, the only map has string keys, and an array cannot be shared between the branches of a fork. The port therefore splits a transfer into its plan, which depends on the two matchings only, and its application to a morphism, and before the forked map of a stage a sequential pass builds a table with one entry per pair of matchings that occurs: a binary trie over 20 bits of a hash of the pair, which is plain data and may be shared, read-only, by all branches. A leaf remembers its pair, and a lookup that finds another pair or nothing computes the plan on the spot, so a collision costs

time and never correctness (two of the laws). That halved the single-thread time (0.53 and 0.56, A/A control within 0.98 to 1.01).

Two further findings were about the language rather than the algorithm. Bend is strict and has no `if: Bool.pick` evaluates both alternatives and `&&` both sides. The port had used them as conditionals, and so ran twelve rounds of list work per surface component whether needed or not, compared matchings for every entry of every visited row, and iterated a connectivity closure sixteen times after it had converged; evaluating one alternative takes a separate function that matches on a Boolean parameter. Five such repairs gave 0.75 of the time. And a third of all reference-count drops then came from counting a list of sources at every pivot candidate; the list now carries its length.

The table of plans also ended the parallel gain. It took the work out of the one part that forks: the map fell to 22 to 25% of the time, and with six threads the two inputs took 0.39 and 3.9 s against 0.36 and 4.07 s with one (at the end: 0.25 and 1.99 s against 0.23 and 2.20 s). What had been running in parallel was the redundant work.

As a process the native recognizer (one thread) beats the Python command line on every input tried up to 30 crossings, because Python needs 28 ms to start (2 ms against 27 ms on an 8-crossing unknot, 34 against 44 ms at 30 crossings); in-process, Python is on a par up to 25 crossings on the same steps, and 1.7 to 4.1 times faster on the two large scans of the table.

11.5 Verdict

The promise holds in the narrow sense and not in the useful one. Writing a fork is trivial and safe. But the scan alternates a parallel map with a Gaussian elimination in which each cancellation changes the rows the next one reads. While the map recomputed every plan for every entry it was 20 to 60% of the time and six threads gave 1.2 to 1.4 overall; once the plans were computed once per pair of matchings, which is the first thing a sequential programmer does, one thread was as fast as six. The cost model (a fork anywhere taxes all sequential code by 1.8 unless it is wrapped as a task) is invisible in the language, and the compiler removes the wrapper unless it is written just so. For this algorithm the measured ranking stays: Rust, then Python, then Bend, the last 1.7 to 4.1 times behind the Python package on large scans, after becoming 4.5 to 29 times faster than its own first version by sequential changes alone.

12 Assessment and recommendations

What was achieved. Seven mutually consistent exact recognizers, the seventh now in an accelerated Python version and a Rust version that agree with each other and with nine independently accelerated variants; a verified and cross-validated body of code for the terminal ball-pattern test and for cocycle-dual normal surfaces; a precise, sourced statement of the conditional $L(g+1)^L$ accounting and of everything a $n^{O(\log n)}$ implementation would still need; and one recognizer whose practical reach is well beyond the 2^n -state cubes.

Obstacles that remain. The six items of Section 5. They are research problems in 3-manifold topology and algorithm design, not coding tasks: the specification of the accelerated hierarchy exists only as a sketch on slides, and the one detailed paper on the subject states that its steps are not specified precisely enough for a running-time estimate. Until that changes, any “ $n^{O(\log n)}$ implementation” would be an exponential search with a mislabelled bound, which is exactly what all six archives refused to deliver.

If work continues. (1) Track the published record: a full version of the 2021 announcement, or the refined Section 10 algorithm with an explicit cost model, would change the picture. (2) Settle the move-set question of Section 3.4 from Dynnikov’s paper, or add shared-corner exchanges to archive 01 and re-run its exhaustive validation. (3) The scanning recognizer now exists in Rust (Section 9); the next step there is to replace the greedy order by a tree-decomposition-based one; the planar separator theorem gives width $O(\sqrt{n})$ for the projection graph, which would make the empirical behaviour subexponential even if unproved. (4) Use the archives’ exact recognizers as oracles for any future geometric implementation, as several of their reports suggest.

References

- [1] Marc Lackenby, *Unknot recognition in quasi-polynomial time*, slides, February 2021, 109 pages, supplied as docs/quasipolynomial-talk.pdf.
- [2] Marc Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026, 54 pages. <https://arxiv.org/abs/2607.23350>.
- [3] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, Publ. Math. IHÉS 113 (2011), 97–208.
- [4] Dror Bar-Natan, *Khovanov’s homology for tangles and cobordisms*, Geom. Topol. 9 (2005), 1443–1499; and *Fast Khovanov homology computations*, J. Knot Theory Ramifications 16 (2007), 243–255.
- [5] Ivan Dynnikov, *Arc-presentations of links: monotonic simplification*, Fund. Math. 190 (2006), 29–76, arXiv:math/0208153.
- [6] Harry M. Markowitz, *The elimination form of the inverse and its application to linear programming*, Management Science 3 (1957), 255–269.
- [7] Mikhail Khovanov, *A categorification of the Jones polynomial*, Duke Math. J. 101 (2000), 359–426.

A Files produced

- reports/01 – reports/06: the extracted archives (flat, one level).
- synthesis/report.tex, synthesis/report.pdf: this report; synthesis/make_tables.py regenerates the tables from synthesis/data/*.json; the cross-validation scripts are in synthesis/data/.
- fast/: the fastunknot package (version 0.2 with the later work of Section 10: the default scanner scan_fast.py and its geometry planar.py), tests and examples; ablation.py with results/ablation.json; the 0.1 benchmark, kept as results/benchmark_0.1.json; and the measuring tools of Section 10, namely perf.py, ab.py and crosscheck.py, with their logs in results/.
- proposals/01 – proposals/09: the nine acceleration proposals, with proposals/README.md comparing their ideas.
- rust/: the Rust crate, cross_check.py, profile.py and results/profile.json.
- synthesis/data/proposals_bench.py and its JSON outputs: the same-machine comparison of all engines.